Week 4 - AWS Service Catalog (portfolios, products, launch constraints, TagOptions)

DOP-C02 Domains 2 and 6. Typical questions:

1. Developers must **self-service launch approved infrastructure without IAM permissions** to the services = **product + portfolio + launch constraint**.
2. **Restrict** instance types / sizes users can pick = **template constraint**.
3. **Standardise tags** on everything users launch = **TagOptions**.
4. **Share** the catalog with all accounts in the organization = **Organizations portfolio sharing** (hub and spoke).
5. Users **restart / back up** their provisioned instances without EC2 permissions = **service actions**.

The big idea: **admins curate** approved IaC (CloudFormation or Terraform) as **products** in **portfolios** with **constraints**; **end users** launch them **self-service**; Service Catalog provisions **with its own launch role**, so users never need broad permissions.

Part 1 - Core concepts

```
Catalog administrator (hub account)
  Portfolio "Approved Compute"        <- access (IAM principals), constraints, TagOptions, sharing
    Product "Standard EC2 Web Server" <- CloudFormation template, versions v1, v2
    Product "Encrypted RDS PostgreSQL"
    Product "Team VPC"
End user (developer)
  Products -> Launch -> parameters / TagOptions
    -> Provisioned product (CloudFormation stack SC-<acct>-pp-xxxx, created with the LAUNCH ROLE)
    -> Update (version / params), Terminate, run service actions
```

Concept	Meaning
Product	IaC template packaged for reuse: **CloudFormation**, **Terraform** (open source via reference engine, or HCP Terraform / Terraform Cloud), **External**; has **versions** (provisioning artifacts)
Product version	**Active / inactive** (inactive can't be launched, existing keep running); guidance **DEFAULT / DEPRECATED** (deprecated hidden from new launches, can still be updated)
Portfolio	**Collection of products** + **who can use them** + **constraints** + **TagOptions** + **sharing**; **a product can be in several portfolios**
Provisioned product	Launched instance, usually **a CloudFormation stack**; **update** or **terminate**
Constraint	Governance rule on a **product within a portfolio** (launch, template, notification, StackSet, tag update)
TagOption	Admin-managed **tag key / value** users select (or auto-applied)
Service action	**SSM document** users run on their provisioned products

Roles and IAM

- Admins: `AWSServiceCatalogAdminFullAccess`.
- End users: `AWSServiceCatalogEndUserFullAccess` (launch, update, terminate) or `AWSServiceCatalogEndUserReadOnlyAccess`; **no EC2 / RDS / IAM permissions needed** with a **launch constraint**.
- **Portfolio access**: associate **IAM users, groups, roles** or **principal names** (role names with wildcards - SSO permission-set roles, shared portfolios).
- **Access level filter**: users see their own provisioned products by default; **Role** / **Account** level shows everything by the role or account.

Part 2 - Products and versions

Item	Detail
Template source	S3 URL, upload, or **repository sync** (GitHub, GitLab, Bitbucket, CodeCommit via **CodeConnections**) - **each commit to the tracked branch creates a new version**
CI/CD	**CodePipeline Service Catalog deploy action**; `CreateProvisioningArtifact` API; `AWS::ServiceCatalog::CloudFormationProduct`; CDK `ProductStack`
Versions	Mark old versions **inactive** or **deprecated**; users **update** provisioned products to the new version (stack update)
Terraform products	Open source needs the **Terraform reference engine** in the hub account; HCP Terraform / Terraform Cloud integration; **launch role** needed
Copy product	`CopyProduct` to another Region
AppRegistry	Registry of **applications** and **attribute groups** linking stacks and resources to an application (myApplications)

Part 3 - Constraints

Constraints apply to a product within a specific portfolio - the same product can have different constraints in different portfolios.

Constraint	What it does	Exam keyword
Launch	**IAM role Service Catalog assumes** to provision, update, terminate; **users need only Service Catalog permissions**	Users have **no permissions** to EC2 / RDS / IAM; **least privilege**
Template	**Rules** (CloudFormation `Rules` syntax) **restricting parameter values**	**Limit** instance types / sizes users can choose
Notification	**SNS topic** receives provisioning **stack events**	**Notify** on launch / failure
StackSet	Deploy the product as a **StackSet** across **multiple accounts and Regions**	One launch -> many accounts / Regions
Tag update (RESOURCE_UPDATE)	**Allow or disallow** users **changing tags** after launch	Users must not change tags

Launch constraint details

```
End user (only servicecatalog end-user permissions)
   | ProvisionProduct
   v
Service Catalog assumes LAUNCH ROLE (trust: servicecatalog.amazonaws.com)
   | cloudformation:* on SC stacks + ec2:RunInstances, iam:PassRole, s3:GetObject (template) ...
   v
CloudFormation stack creates the resources
```

- Trust policy: `servicecatalog.amazonaws.com`.
- Permissions: **CloudFormation + every service in the template** + `s3:GetObject` on the template bucket + `iam:PassRole` if the template passes roles.
- The **admin creating the constraint needs** `iam:PassRole` on the role.
- **One launch constraint per product per portfolio.**
- **Without a launch constraint**, Service Catalog uses the **end user's credentials** (user needs all resource permissions).
- **Shared portfolios**: `LocalRoleName` (role name, not ARN) **resolves to the role with that name in each recipient account**.
- Updates and terminations also use the launch role.

Template constraint example

```
{
  "Rules": {
    "AllowedInstanceTypes": {
      "Assertions": [{
        "Assert": { "Fn::Contains": [ ["t3.micro", "t3.small"], { "Ref": "InstanceType" } ] },
        "AssertDescription": "Only t3.micro or t3.small are allowed"
      }]
    }
  }
}
```

Part 4 - TagOptions and automatic tags

Feature	Detail
TagOption library	Admin-defined **key / value pairs** (`CostCenter=100`, `CostCenter=200`, `Env=dev`)
Association	With **portfolios** (all products) or **individual products**
At launch	Several values for a key = user **must pick** from the list (no free text); **single value = applied automatically**
Result	Tags on the **provisioned product**, **propagated to stack resources** - consistent cost allocation and ABAC
Inactive	TagOptions can be **deactivated**
Sharing	Optionally **share TagOptions** with a portfolio share
AutoTags	`aws:servicecatalog:portfolioArn`, `productArn`, `provisionedProductArn`, `provisioningPrincipalArn`, `provisioningArtifactIdentifier`

TagOptions = consistent tags on Service Catalog launches; **Organizations tag policies** = standardise org-wide; **SCP with** `aws:RequestTag` = mandatory tags on every creation path.

Part 5 - Sharing portfolios

Method	How	Notes
Account-to-account	Share with an **account ID**; recipient **imports**	**Imported portfolio**: recipient **can't edit products**, **can add its own principals and constraints**; hub product updates **appear automatically**
Organizations sharing	Share with **org, OU or account**; **no manual import**	**Trusted access**; optional **delegated administrator** (hub isn't the management account)
Share options	**Share TagOptions**, **share principals** (principal names like `role/Developer*`)	Principal sharing + `LocalRoleName` = **zero-touch hub and spoke**

Hub-and-spoke pattern:

1. **Hub account** owns portfolios and products (versions published by a pipeline).
2. Portfolios shared to **spoke accounts / OUs** via Organizations.
3. Each spoke has `SCLaunchRole` (deployed by **StackSets**) and developer roles (SSO permission sets).
4. **Launch constraint** `LocalRoleName: SCLaunchRole` + **shared principal names** = developers everywhere launch products **in their own account**.

Part 6 - Service actions, lifecycle, integrations

Service actions

- **SSM documents** (usually Automation: `AWS-RestartEC2Instance`, `AWS-StopEC2Instance`, `AWS-CreateImage`, custom) **associated with product versions**.
- **End users run them** on their provisioned products **without EC2 or SSM permissions**.
- "Developers must restart / back up their Service Catalog EC2 without EC2 console access" = **service actions**.

Lifecycle

Operation	Notes
Launch (`ProvisionProduct`)	Product, version, parameters, TagOptions; **plans** (`CreateProvisionedProductPlan`) preview changes (change set)
Update	New version / parameters -> stack update (launch role)
Terminate	Deletes the stack (DeletionPolicy applies)
Import	`ImportAsProvisionedProduct` for existing CloudFormation stacks
Owner change	`UpdateProvisionedProductProperties`
Monitoring	Notification constraint, EventBridge, CloudTrail, CloudFormation events
Budgets	**AWS Budgets** associated with portfolios / products
From CloudFormation	`AWS::ServiceCatalog::CloudFormationProvisionedProduct`

Integrations

Integration	Detail
Control Tower	**Account Factory is a Service Catalog product**; **AFC blueprints** are SC products in a hub account
CloudFormation	`Portfolio`, `CloudFormationProduct`, `LaunchRoleConstraint`, `LaunchTemplateConstraint`, `LaunchNotificationConstraint`, `StackSetConstraint`, `ResourceUpdateConstraint`, `PortfolioPrincipalAssociation`, `PortfolioProductAssociation`, `TagOption`, `TagOptionAssociation`, `PortfolioShare`, `ServiceAction`
CDK	`aws-servicecatalog` (Portfolio, CloudFormationProduct, **ProductStack**)
CodePipeline	**Service Catalog deploy action** for new versions
ITSM	**ServiceNow** and **Jira Service Management** connectors
Organizations	Trusted access, org / OU sharing, delegated admin

Requirement	Best choice
Users **self-service** launch **pre-approved** stacks, **no direct permissions**	**Service Catalog**
Push a baseline into **every** account automatically	**StackSets** / CfCT
Landing zone + account vending	**Control Tower** (Account Factory = SC product)
Developers deploy app code	CodePipeline / CDK / SAM
Preventive guardrails	SCPs / Control Tower controls
Platform templates for new designs	**Service Catalog** (AWS Proton is being discontinued)

Part 7 - Exam traps

Trap	Truth
Give developers EC2 / RDS / IAM permissions to launch products	**Launch constraint**
AccessDenied with a launch constraint	Launch role missing service / CloudFormation / `s3:GetObject` / `iam:PassRole`, or **trust not** `servicecatalog.amazonaws.com`
Admin can't create launch constraint	Needs `iam:PassRole` on the role
Two launch constraints on one product in one portfolio	**One per product per portfolio**
One launch constraint for many spoke accounts	`LocalRoleName`
Limit instance types	**Template constraint**
Notify on launch / failure	**Notification constraint**
One launch to many accounts / Regions	**StackSet constraint**
Stop users editing tags after launch	**Tag update constraint**
Consistent tag values on launches	**TagOptions**
TagOptions make tags mandatory org-wide	**No**; **SCP +** `aws:RequestTag`
Recipient edits imported product	**Can't**; adds own principals and constraints
Share with all accounts in an OU, no imports	**Organizations sharing** (+ delegated admin)
Developers in every account get access automatically	**Share principals** (principal names)
Restart provisioned EC2 without EC2 access	**Service actions**
New version on every Git commit	**Repo sync (CodeConnections)** or **CodePipeline SC action**
Stop launches of old version, keep existing	**Inactive / deprecated**
Preview changes before update	**Provisioned product plan**
Existing stacks under Service Catalog	`ImportAsProvisionedProduct`
Terraform products	**Reference engine** or HCP Terraform + **launch role**
Push baseline to every account	**StackSets / CfCT**, not Service Catalog
Account Factory relationship	**Account Factory is an SC product**

Part 8 - Hands-on lab (about 60 minutes, very low cost)

Lab 1 - Product and portfolio

1. `web.yaml` in S3: parameters `InstanceType` (default `t3.micro`) and `LatestAmi` (`AWS::SSM::Parameter::Value<AWS::EC2::Image::Id>` with the AL2023 public parameter); one EC2 instance + SG without inbound rules; output instance ID.
2. **Create product** (CloudFormation) `Standard Web Server`, version `v1`, S3 template URL.
3. **Create portfolio** `Approved Compute` and add the product.

Lab 2 - Launch constraint and end user

1. Role `SCLaunchRole`: trust `servicecatalog.amazonaws.com`; permissions `cloudformation:*` on `SC-*` stacks, `ec2:*` (lab), `ssm:GetParameters`, `s3:GetObject` on the template bucket.
2. Portfolio -> Constraints -> **Launch** with `SCLaunchRole`.
3. User / role `Developer` with **only** `AWSServiceCatalogEndUserFullAccess`; associate it with the portfolio.
4. As `Developer`: **Launch** succeeds without EC2 permissions; check stack `SC-...-pp-...`; EC2 console launch is **denied**.

Lab 3 - Template constraint

1. Template constraint with the Rules JSON above.
2. As `Developer` launch with `m5.large`: **rejected with the AssertDescription**.

Lab 4 - TagOptions

1. TagOptions `CostCenter=100`, `CostCenter=200`, `Env=dev`; associate with the portfolio.
2. Launch: **must pick CostCenter**, `Env=dev` auto-applied; check EC2 tags incl. `aws:servicecatalog:*`.
3. **Tag update constraint** (not allowed): updating tags is blocked.

Lab 5 - Versions, notifications, service actions

1. Add **v2** (detailed monitoring); as `Developer` **update** to v2; mark **v1 deprecated**.
2. **Notification constraint** with an SNS email topic; launch and receive stack events.
3. **Service action** from `AWS-RestartEC2Instance` (`InstanceId` target), associate with v2; `Developer` runs **Restart** without EC2 permissions.

Lab 6 - Sharing (optional)

1. Enable **Organizations access**; share with an OU, **share TagOptions** and **principals**; launch constraint `LocalRoleName: SCLaunchRole`.
2. Deploy `SCLaunchRole` and `Developer` in the spoke (StackSets); developers launch **in their own account**.

Cleanup: terminate provisioned products; delete service actions, constraints, TagOptions, portfolio, product, IAM roles / users, SNS topic, S3 template.

Part 9 - Check yourself

Questions

1. Product vs product version vs portfolio vs provisioned product?
2. What does a launch constraint do, and what permissions do end users still need?
3. AccessDenied with a launch constraint - three things to check?
4. Why use `LocalRoleName` in a launch constraint?
5. How do you restrict users to t3.micro and t3.small?
6. Which constraint sends stack events, and which deploys to many accounts / Regions?
7. How do TagOptions work at launch, and with a single-value key?
8. TagOptions vs Organizations tag policy vs SCP `aws:RequestTag`?
9. Prevent users changing tags on provisioned products?
10. What can and can't a recipient do with an imported portfolio?
11. Share with every account in an OU and give developers access automatically?
12. Restart / back up Service Catalog EC2 without EC2 permissions?
13. Publish a new version automatically when the template changes in Git?
14. Stop new launches of an outdated version without breaking existing products?
15. Service Catalog vs StackSets - pull vs push? How does Control Tower use Service Catalog?

Answers

1. **Product** = packaged IaC template; **version** = revision; **portfolio** = products + access + constraints + TagOptions + sharing; **provisioned product** = launched instance (stack).
2. Service Catalog **assumes the launch role** to provision / update / terminate; users need only **Service Catalog end-user permissions**.
3. Launch role **permissions** (template services, CloudFormation, `s3:GetObject`, `iam:PassRole`), **trust** `servicecatalog.amazonaws.com`, parameter / resource scope.
4. Works across **shared portfolios**: the name resolves to the **local role in each account**.
5. **Template constraint** with Rules (`Fn::Contains` on InstanceType).
6. **Notification constraint** (SNS); **StackSet constraint**.
7. User **picks from admin values**; **single value auto-applied**; tags propagate to resources.
8. TagOptions for SC launches; tag policies standardise org-wide; **SCP makes tags mandatory everywhere**.
9. **Tag update (RESOURCE_UPDATE) constraint** not allowed.
10. Can **add principals and constraints**, receives product updates; **can't edit products**.
11. **Organizations sharing** to the OU (trusted access, delegated admin) with **share principals** + `LocalRoleName`.
12. **Service actions** (SSM documents).
13. **Repo sync with CodeConnections** or **CodePipeline Service Catalog action**.
14. Mark it **inactive / deprecated**.
15. Service Catalog = **self-service pull**; StackSets = **push**; **Account Factory (and AFC blueprints) are SC products**.

Quick memory summary

- **Service Catalog** = governed **self-service** IaC: **products** (CloudFormation / Terraform / External, **versions**) in **portfolios** (IAM principals / principal names) launched as **provisioned products** (stacks).
- **Constraints** (per product per portfolio): **launch** (role, trust `servicecatalog.amazonaws.com`, one per product per portfolio, `LocalRoleName`), **template** (Rules), **notification** (SNS), **StackSet**, **tag update**.
- **TagOptions**: library on portfolios / products, pick from list, single value auto, auto `aws:servicecatalog:*` tags; SC launches only (SCP for mandatory).
- **Sharing**: account (import; can't edit products) or **Organizations** (OU / org, delegated admin, share TagOptions + principals) = **hub and spoke**.
- **Service actions** (SSM), repo sync / CodePipeline for versions, deprecate / inactive, plans, import, budgets, ServiceNow / Jira; **Account Factory is an SC product**.